LiveKit Load-Test Framework

What it is, what it achieves today, and the path to 100 / 500 / 1,000 / 2,500 / 10,000 users

Status note: this document is based on a review of the code and the team's architecture discussion. No load test has been run yet, so capacity figures are **estimates** to be confirmed by the first measured ramp test.

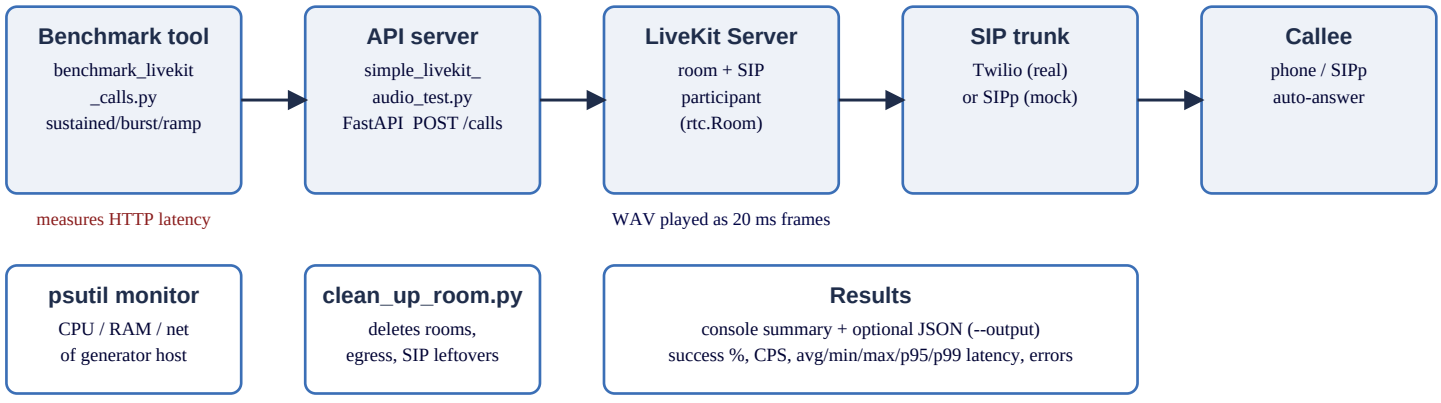
1. Executive summary

- The framework generates SIP call load against LiveKit and reports throughput, API latency, errors and load-generator host resources.
- It already supports three test shapes: **sustained**, **burst** and **ramp-up**.
- Estimated capacity today: about **50-150 concurrent calls per machine** (single Python process).
- It is a good base for the **100-user tier**. Larger tiers need distributed generators, agent-aware measurement and Prometheus/Grafana integration.
- Biggest gap: it measures how fast the API *accepts* a call, not whether the call connected or how fast the AI agent responded.

2. Components

File	Role
simple_livekit_audio_test.py	FastAPI server (POST /calls, GET /calls/{id}, DELETE, upload). Each call creates a LiveKit room, dials a SIP participant through the trunk, joins as an rtc.Room client and streams a WAV file in 20 ms frames.
benchmark_livekit_calls.py	Load generator. Sends POST /calls at a target rate using asyncio + aiohttp, collects results, samples host metrics with psutil, prints a summary and saves JSON.
clean_up_room.py	Removes leftover rooms, egress jobs and SIP resources after a test (run with pytest).
Makefile / pyproject.toml	Install (Poetry, Python 3.11-3.12) and one-line run targets.

3. How it works



The SIP caller ID (Twilio *from* number) comes from the LiveKit outbound trunk; the *to* number is sent per call in E.164 format. For large tests the trunk should point to a mock SIP endpoint (SIPp auto-answer with RTP echo) instead of real phones.

4. Test modes

Mode	What it does	Use it to
Sustained	Fixed calls-per-second for a set duration, with a concurrency cap	Check stability at a target load
Burst	N calls as fast as possible, limited by a semaphore	Find peak acceptance capacity
Ramp-up	Linearly increases CPS to a maximum, then holds	Find the breaking point

5. What the framework achieves today

5.1 Metrics captured

Metric	Covered?	Note
Total / successful / failed requests, success rate	Yes	Based on HTTP 200 from POST /calls
Achieved calls per second	Yes	Total requests / test duration
API latency avg, min, max, p95, p99	Yes	Time for the API to accept the request, not call setup time
Error breakdown	Yes	Grouped by HTTP status or exception message
Generator CPU, memory, disk, network	Yes	psutil every 1 s, generator host only
Per-call state (pending / active / progress)	Partial	Exposed by GET /calls/{id}, not used by the benchmark
Result export	Partial	Console + JSON; no live dashboard

5.2 Estimated capacity by tier

Concurrent users	Current framework	Reason
100	Achievable	One good machine (8+ cores); verify audio timing does not drift
500	Marginal	Needs 3-5 generator processes / machines
1,000	No	Single event loop saturates; needs distributed generators
2,500	No	Distributed generators + higher Twilio / provider limits
10,000	No	Infrastructure + budget project (see section 8)

5.3 Strengths

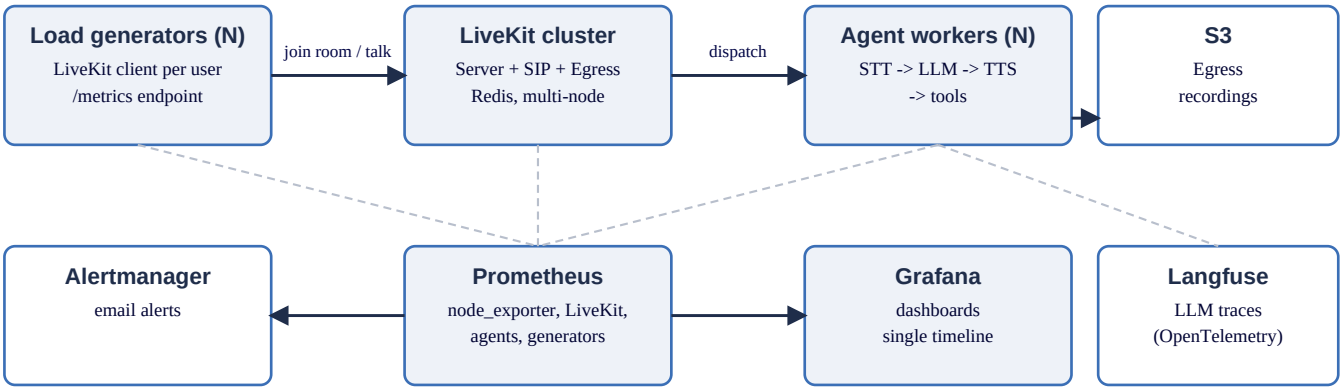
- Simple, readable Python; easy for the team to extend.
- Three standard load shapes built in.
- Percentile latency and error breakdown already implemented.
- Exercises the real LiveKit + SIP path, including audio streaming.
- Cleanup utility avoids leaked rooms between runs.

6. Limitations and known issues

Area	Issue	Impact
Scale	Single uvicorn process; every call is an rtc.Room + 50 frames/s in one event loop	Caps at roughly 100-150 calls per machine
Measurement	Latency = API acceptance only; call connect and agent response not measured	A 200 response can hide failed calls
Bug	Sustained test drops results of tasks that finished during the loop	Under-counts completed calls
Bug	Burst test creates its task list twice; first batch never awaited	Extra unmeasured load

Area	Issue	Impact
Overhead	tracemalloc and per-call console prints in the server	Lower capacity, distorted results
Observability	No Prometheus endpoint; results only at end of run	Cannot correlate with server/agent metrics in Grafana
Scope	SIP calls only; does not talk to the AI agent (STT/LLM/TTS)	Agent latency (TTFT, TTFB, end-to-end) not covered
Packages	aiohttp, psutil, pytest used but not declared in pyproject.toml	Possible install errors

7. Target architecture (from team discussion)



7.1 Metrics we need to cover

Layer	Metrics	Source
Load generator	Users started / connected / failed, call setup time, agent response time, audio frame drift	New /metrics endpoint (prometheus_client)
AI agent	LLM TTFT, STT latency, TTS TTFB, end-of-utterance delay, end-to-end response time	livekit-agents metrics_collected -> Prometheus; traces -> Langfuse
LiveKit Server / SIP / Egress	Rooms, participants, packet loss, jitter, bandwidth, SIP errors	Built-in prometheus_port (self-hosted)
Infrastructure	CPU, memory, network per VM	node_exporter
External providers	STT / LLM / TTS errors and 429 rate limits	Agent metrics + logs
Alerting	e.g. CPU > 80 %, TTFT p95 > 2 s, error rate > 1 %	Alertmanager -> email

Langfuse has its own UI and is not a direct Grafana data source: numbers go to Prometheus/Grafana, per-conversation traces stay in Langfuse.

8. Roadmap to the targets

Tier	Work required	Setup
100	Fix two benchmark bugs, remove tracemalloc, poll call status, add /metrics	1 generator machine, SIPp mock, 5-10 real Twilio calls
500	Multi-process runner, Grafana dashboard	3-5 generator processes / machines
1,000	Agent-aware generator (LiveKit client sends WAV, waits for agent reply)	Distributed generators, separate agent-worker machines
2,500	Raise STT / LLM / TTS and Twilio quotas; LiveKit multi-node	LiveKit cluster + Redis

Tier	Work required	Setup
10,000	Capacity model from earlier tiers; budget and provider approval	Dozens-hundreds of agent machines, ~70-100 generator machines at current efficiency

Approach: measure one unit (one agent machine, one LiveKit node) at 100 users, scale tier by tier, fix the first bottleneck each time, then size 10,000 from the measured numbers.

9. Decisions needed from the team

- Is 10,000 **concurrent** users or 10,000 users **per day**? This changes infrastructure size by 10-20x.
- First test target: WebRTC users talking to the AI agent, or SIP phone calls?
- Twilio account: trial (only verified numbers) or paid? Current CPS / concurrency limits?
- LiveKit: self-hosted (Prometheus available) or LiveKit Cloud?
- Approval to use SIPp as the mock call endpoint for large tiers.

10. Requirements to run

- Python 3.11 or 3.12 (3.13 not supported: audioop removed), Poetry.
- Packages: livekit, livekit-agents 1.1.1, fastapi, uvicorn, numpy, python-multipart, plus aiohttp, psutil, pytest.
- .env: LIVEKIT_URL, LIVEKIT_API_KEY, LIVEKIT_API_SECRET, LIVEKIT_SIP_TRUNK_ID.
- A WAV file (8 kHz, mono, 16-bit) in uploads/.
- Run: `make benchmark-server-up`, then `benchmark_livekit_calls.py --ramp-up ... --output results.json`.